

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 6: TRANSFORM-AND-CONQUER (INSTANCE SIMPLIFICATION)

Abdullah Bal, PhD

Objectives



Recap

Multiplication of large integers
Strassen's matrix multiplication
Closest-Pair Problem by DAC



Transform-and-
Conquer

Instance
Simplification



Presorting



Gaussian
Elimination

Transform and Conquer

- Instance simplification: This group of techniques solves a problem by a transformation to a simpler/more convenient instance of the same problem.
- Representation change: A different representation of the same instance
- Problem reduction: A different problem for which an algorithm is already available

Instance simplification

Presorting

Presorting

- Solve a problem's instance by transforming it into another simpler/easier instance of the same problem

Presorting

- Many problems involving lists are easier **when list is sorted**, e.g.
- Searching
- Computing the median (selection problem)
- Checking if all elements are distinct (element uniqueness)

Also:

- Topological sorting helps solving some problems for dags.
- Presorting is used in many geometric algorithms.

How fast can we sort ?

- Efficiency of algorithms involving sorting depends on **efficiency of sorting**.
- Theorem $\lceil \log_2 n! \rceil \approx n \log_2 n$ comparisons are necessary in the worst case to sort a list of size n by any comparison-based algorithm.
- Note: About $n \log_2 n$ comparisons are also sufficient to sort array of size n (by mergesort).

Stirling formula

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \text{ for large values of } n.$$

Example 1: Element Uniqueness with presorting

Bruteforce

- Checking element uniqueness in an array
- The **bruteforce** algorithm compared pairs of the array's elements until either two equal elements were found or no more pairs were left.
- Its worst-case efficiency was in $\Theta(n^2)$.

Example 1: Element Uniqueness with presorting

Presorting

- Alternatively, we can sort the array first and then check only its consecutive elements:
 - if the array has equal elements, a pair of them must be next to each other, and vice versa.

Example 1: Element Uniqueness with presorting

ALGORITHM *PresortElementUniqueness*($A[0..n - 1]$)

//Solves the element uniqueness problem by sorting the array first

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Returns “true” if A has no equal elements, “false” otherwise

sort the array A

for $i \leftarrow 0$ **to** $n - 2$ **do**

if $A[i] = A[i + 1]$ **return false**

return true

Example 1: Element Uniqueness with presorting

- The running time of this algorithm is the sum of the time spent on sorting and the time spent on checking consecutive elements.

$$T(n) = T_{\text{sort}}(n) + T_{\text{scan}}(n)$$

- Since the former (sorting) requires at least $n \log n$ comparisons and the latter (scanning) needs no more than $n - 1$ comparisons, it is the sorting part that will determine the overall efficiency of the algorithm.

Example 1: Element Uniqueness with presorting

- So, if we use a quadratic sorting algorithm here, the entire algorithm will not be more efficient than the brute-force one.
- But if we use a good sorting algorithm, such as merge sort, with worst-case efficiency in $\Theta(n \log n)$, the worst-case efficiency of the entire presorting-based algorithm will be also in $\Theta(n \log n)$:

Example 1: Element Uniqueness with presorting

- Presorting-based algorithm

- Stage 1: sort by efficient sorting algorithm (e.g. merge sort)
- Stage 2: scan array to check pairs of adjacent elements
- Efficiency: $\Theta(n \log n) + O(n) = \Theta(n \log n)$

- Brute force algorithm

- Compare all pairs of elements
- Efficiency: $O(n^2)$

Example 2: Computing a Mode

- A *mode* is a value that **occurs most often** in a given list of numbers.
- For example, for 5, 1, 5, 7, 6, 5, 7, **the mode is 5**.
- If several different values occur most often, any of them can be considered a mode.

Example 2: Computing a Mode

Bruteforce

- The **brute-force approach** to computing a mode would scan the list and compute the frequencies of all its distinct values, then find the value with the largest frequency.
- In order to implement this idea, we can **store the values already encountered**, along with **their frequencies**, in a **separate** list.
- On each iteration, the i^{th} element of the original list is compared with the values already encountered by traversing this **auxiliary list**.
- If a matching value is found, **its frequency is incremented**; otherwise, the current element is added to the list of distinct values seen so far **with a frequency of 1**.

Example 2: Computing a Mode

- The worst-case input for this algorithm is a list with no equal elements.
- For such a list, its i^{th} element is compared with $i - 1$ elements of the **auxiliary list of distinct values** seen so far before being added to the list with a frequency of 1.
- As a result, the worst-case number of comparisons made by this algorithm in creating the frequency list is $\Theta(n^2)$
- The **additional $n - 1$ comparisons** needed to find the largest frequency in the auxiliary list do **not change the quadratic worst-case** efficiency class of the algorithm.
- Efficiency: $\Theta(n^2)$

Example 2: Computing a Mode

Presorting

- As an alternative, let us first sort the input.
- Then all equal values will be adjacent to each other.
- To compute the mode, all we need to do is to find the longest run of adjacent equal values in the sorted array.
- The running time of the algorithm will be dominated by the time spent on sorting since the remainder of the algorithm takes linear time.
- Consequently, with a $n \log n$ sort, this method's worst-case efficiency will be in a better asymptotic class than the worst-case efficiency of the brute-force algorithm.
- Efficiency: $\Theta(n \log n)$

Example 2: Computing a Mode

ALGORITHM *PresortMode*($A[0..n - 1]$)

//Computes the mode of an array by sorting it first
//Input: An array $A[0..n - 1]$ of orderable elements
//Output: The array's mode
sort the array A
 $i \leftarrow 0$ //current run begins at position i
 $modefrequency \leftarrow 0$ //highest frequency seen so far
while $i \leq n - 1$ **do**
 $runlength \leftarrow 1$; $runvalue \leftarrow A[i]$
 while $i + runlength \leq n - 1$ **and** $A[i + runlength] = runvalue$
 $runlength \leftarrow runlength + 1$
 if $runlength > modefrequency$
 $modefrequency \leftarrow runlength$; $modevalue \leftarrow runvalue$
 $i \leftarrow i + runlength$
return $modevalue$

Example 3: Searching Problem

- Problem: Search for a given K in $A[0..n-1]$

Bruteforce

- The brute-force solution here is sequential search, which needs n comparisons in the worst case.
- Efficiency: $\Theta(n)$

Example 3: Searching Problem

- Presorting

- Presorting-based algorithm:

 - Stage 1: Sort the array by an efficient sorting algorithm

 - Stage 2: Apply binary search

- Assuming the most efficient $n \log n$ sort, the total running time of such a searching algorithm in the worst case will be

$$T(n) = T_{\text{sort}}(n) + T_{\text{search}}(n)$$

- Efficiency: $\Theta(n \log n) + \Theta(\log n) = \Theta(n \log n)$

Example 3: Searching Problem

- Brute force algorithm

 - Efficiency: $\Theta(n)$

- Presorting-based algorithm

 - Efficiency: $\Theta(n \log n) + \Theta(\log n) = \Theta(n \log n)$

- Good or bad?

- Why do we have our dictionaries, telephone directories, etc. sorted? If we are to search in the same list more than once, the time spent on **sorting might well be justified.**

Instance simplification

Gaussian Elimination

Gaussian Elimination

- We are certainly familiar with systems of two **linear equations** in two unknowns:

$$\begin{aligned}a_{11}x + a_{12}y &= b_1 \\ a_{21}x + a_{22}y &= b_2.\end{aligned}$$

- Unless the coefficients of one equation are proportional to the coefficients of the other, the system has a **unique solution**.
- The standard method for finding this solution is to use either equation to express one of the variables **as a function of the other** and then **substitute** the result into the other equation, yielding a linear equation whose solution is then used to find the value of the second variable.

Gaussian Elimination (cont.)

- In many applications, we need to solve a system of n equations in n unknowns:

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\&\vdots \\a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n\end{aligned}$$

where n is a large number.

- Theoretically, we can solve such a system by generalizing the substitution method for solving systems of two linear equations.
- However, the resulting algorithm would be extremely cumbersome.

Gaussian Elimination (cont.)

- There is a much **more elegant algorithm** for solving systems of linear equations called ***Gaussian elimination***.
- The idea of Gaussian elimination is to transform a system of n linear equations in n unknowns to an equivalent system (i.e., a system with the same solution as the original one) with an **upper- triangular coefficient matrix**, a matrix with **all zeros below its main diagonal**:

$$\begin{array}{l} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{array} \implies \begin{array}{l} a'_{11}x_1 + a'_{12}x_2 + \cdots + a'_{1n}x_n = b'_1 \\ a'_{22}x_2 + \cdots + a'_{2n}x_n = b'_2 \\ \vdots \\ a'_{nn}x_n = b'_n. \end{array}$$

- In matrix notations, we can write this as $Ax = b \implies A'x = b'$,

Gaussian Elimination (cont.)

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & & & \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}, \quad A' = \begin{bmatrix} a'_{11} & a'_{12} & \dots & a'_{1n} \\ 0 & a'_{22} & \dots & a'_{2n} \\ \vdots & & & \\ 0 & 0 & \dots & a'_{nn} \end{bmatrix}, \quad b = \begin{bmatrix} b'_1 \\ b'_2 \\ \vdots \\ b'_n \end{bmatrix}.$$

- We can easily solve the system with an upper-triangular coefficient matrix by back substitutions as follows.

Gaussian Elimination (cont.)

- First, we can immediately find the value of x_n from the last equation;
- Then we can substitute this value into the next to last equation to get x_{n-1} , and so on,
- Until we substitute the known values of the last $n - 1$ variables into the first equation, from which we find the value of x_1 .

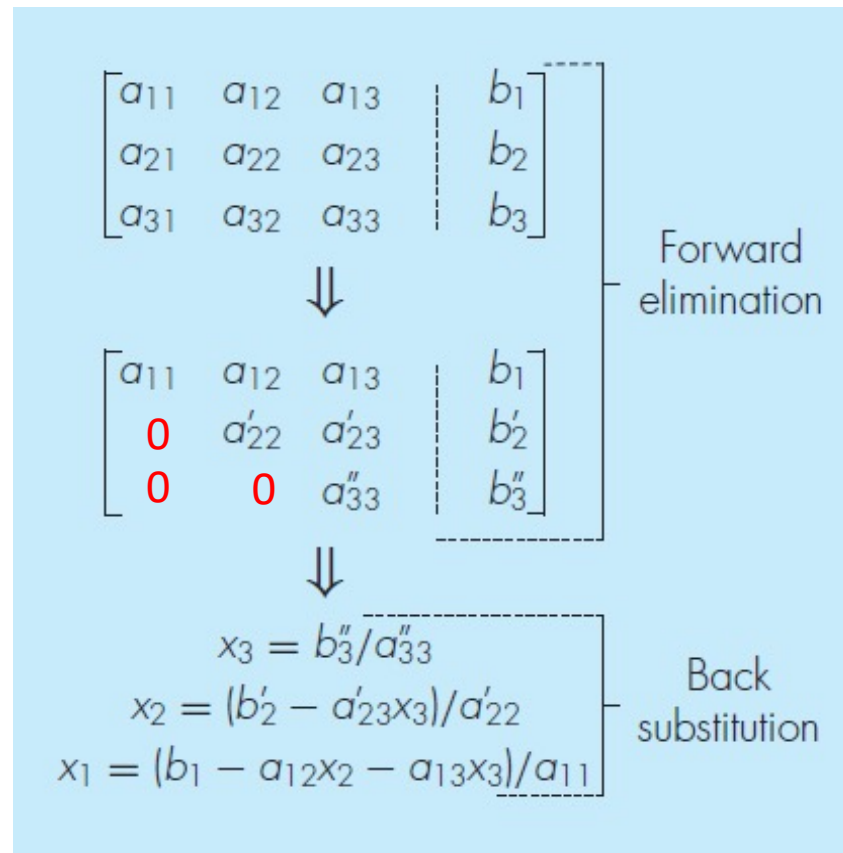
$$\begin{aligned}a'_{11}x_1 + a'_{12}x_2 + \cdots + a'_{1n}x_n &= b'_1 \\a'_{22}x_2 + \cdots + a'_{2n}x_n &= b'_2 \\&\vdots \\a'_{nn}x_n &= b'_n.\end{aligned}$$

Gaussian Elimination (cont.)

- How can we get from a system with an arbitrary coefficient matrix A to an equivalent system with an upper-triangular coefficient matrix A' ?
- We can do that through a series of the so-called *elementary operations*:
 - exchanging two equations of the system replacing an equation with its nonzero multiple
 - replacing an equation with a sum or difference of this equation and some multiple of another equation
 - Since no elementary operation can change a solution to a system, any system that is obtained through a series of such operations will have the same solution as the original one.

Gaussian Elimination (cont.)

- Solve the system by Gaussian elimination.



Gaussian Elimination (cont.)

- Let us see how we can get to a system with an upper-triangular matrix.
- First, we use a_{11} as a *pivot* to make all x_1 coefficients zeros in the equations below the first one.
- Specifically, we replace the second equation with the difference between it and the first equation multiplied by a_{21}/a_{11} to get an equation with a zero coefficient for x_1 .

Gaussian Elimination (cont.)

- Doing the same for **the third, fourth, and finally n th equation**—with the multiples a_{31}/a_{11} , a_{41}/a_{11} , $\dots$, a_{n1}/a_{11} of the first equation, respectively—makes all the coefficients of x_1 below the first equation zero.
- Then we get rid of all the coefficients of x_2 by subtracting an appropriate multiple of the second equation from each of the equations below the second one.
- Repeating this elimination for each of the first $n - 1$ variables **ultimately yields a system with an upper-triangular coefficient matrix.**
- We can operate with just a system's coefficient matrix augmented, as its $(n + 1)^{\text{st}}$ column, with the equations' right-hand side values.

Example 1: Gaussian Elimination

○ Solve the system by Gaussian elimination.

$$2x_1 - x_2 + x_3 = 1$$

$$4x_1 + x_2 - x_3 = 5$$

$$x_1 + x_2 + x_3 = 0.$$

$$\left[\begin{array}{ccc|c} 2 & -1 & 1 & 1 \\ 4 & 1 & -1 & 5 \\ 1 & 1 & 1 & 0 \end{array} \right] \begin{array}{l} \\ \text{row 2} - \frac{4}{2} \text{ row 1} \\ \text{row 3} - \frac{1}{2} \text{ row 1} \end{array}$$

$$\left[\begin{array}{ccc|c} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & \frac{3}{2} & \frac{1}{2} & -\frac{1}{2} \end{array} \right] \text{row 3} - \frac{1}{2} \text{ row 2}$$

$$\left[\begin{array}{ccc|c} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & 0 & 2 & -2 \end{array} \right]$$

Example 1: Gaussian Elimination

Augmented Matrix

Upper-triangular coefficient matrix.

$$\left[\begin{array}{ccc|c} 2 & -1 & 1 & 1 \\ 4 & 1 & -1 & 5 \\ 1 & 1 & 1 & 0 \end{array} \right] \rightarrow \left[\begin{array}{ccc|c} 2 & -1 & 1 & 1 \\ 0 & 3 & -3 & 3 \\ 0 & 0 & 2 & -2 \end{array} \right]$$

Example 1: Gaussian Elimination

○ Now we can obtain the solution by back substitutions:

$$x_3 = (-2)/2 = -1,$$

$$x_2 = (3 - (-3)x_3)/3 = 0,$$

$$x_1 = (1 - x_3 - (-1)x_2)/2 = 1.$$

Pseudocode of Gaussian Elimination

- Stage 1: Reduction to an upper-triangular matrix

```
for  $i \leftarrow 1$  to  $n-1$  do
  for  $j \leftarrow i+1$  to  $n$  do
    for  $k \leftarrow i$  to  $n+1$  do
       $A[j, k] \leftarrow A[j, k] - A[i, k] * A[j, i] / A[i, i]$ 
```

- Stage 2: Back substitutions

```
for  $j \leftarrow n$  down to  $1$  do
   $t \leftarrow 0$ 
  for  $k \leftarrow j+1$  to  $n$  do
     $t \leftarrow t + A[j, k] * x[k]$ 
   $x[j] \leftarrow (A[j, n+1] - t) / A[j, j]$ 
```

- Efficiency: $\Theta(n^3) + \Theta(n^2) = \Theta(n^3)$

Summary

- Transform-and-Conquer
 - Instance simplification
 - Representation Change
 - Problem Reduction
- Instance Simplification
- Presorting
- Gaussian Elimination

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.